RENT MY CAR · LEERUITKOMST 1

AI-log

Logboek van het AI-gebruik bij het ontwerpen, bouwen en testen van de Web API

Module	AI-powered software development
Opleiding	Informatica, jaar 2 (Avans Hogeschool)
Student	Thimo Harte
Versie	25 oktober 2026
Repository	github.com/ThimoHarte/RentMyCar

Inhoud

AI-log 3

2026-10-05 – Analyse, Opstart en Architectuur 3
2026-10-06 – Realisatie: Eerste Code & Domein 3
2026-10-09 – Database Setup & Repositories 3
2026-10-12 – Business Logic & Services 4
2026-10-15 – API Endpoints & Ktor Routing 4
2026-10-19 – Testen & Kwaliteitsborging 4
2026-10-22 – Beveiliging en Authenticatie 4
2026-10-24 – Kwaliteitsslag & Refactoring 5
2026-10-25 – Code Integratie, Documentatie & Oplevering 5

AI-log

In dit logboek houd ik bij op welke momenten ik AI heb gebruikt tijdens het project, wat ik heb gevraagd, wat de AI voorstelde, hoe ik dat heb gecontroleerd en wat ik uiteindelijk heb besloten. Hoofdstuk 06 (productverantwoording) vat dit logboek samen.

Gebruikte tool: Claude Code (model: Claude Opus 5.5), in de terminal naast IntelliJ IDEA.

Afspraken met de AI:

- Eerst overleggen, pas iets maken als ik daarom vraag.
- De code zo eenvoudig mogelijk houden, zodat ik alles zelf kan uitleggen.
- De AI legt uit wat het voorstelt; ik neem alleen over wat ik begrijp.
- Commits en pushes doe ik zelf.

2026-10-05 – Analyse, Opstart en Architectuur

Wat ik vroeg: Een samenvatting van de eisen uit de Brightspace-documenten en hulp bij het bepalen van de scope.

Wat de AI voorstelde: De AI las de documenten en stelde voor om de autoverhuur te automatiseren met GPS-coördinaten en kluiscodes voor de sleutels. Voor de opslag stelde de AI drie opties voor: in-memory, H2 of PostgreSQL.

Mijn beslissing: Ik heb de GPS en kluiscodes afgewezen omdat dit de scope onnodig complex maakt. Ik koos voor een model zoals SnappCar (handmatige sleuteloverdracht). Qua database koos ik voor H2 met JetBrains Exposed, omdat relationele databases een leerdoel zijn.

Ontwerp: Ik heb het domein ontworpen. De AI hielp me met de juiste PlantUML-syntax, waarna ik zelf de klassendiagrammen heb uitgetypt.

2026-10-06 – Realisatie: Eerste Code & Domein

Wat ik vroeg: Uitleg over hoe ik het beste drie soorten auto's (benzine, elektrisch, waterstof) kon modelleren in Kotlin, en hulp bij de actuele brandstofprijzen.

Wat de AI voorstelde: De AI legde het concept van een sealed class uit en adviseerde om alle eigenschappen als val (immutable) te definiëren met controles in een init-blok. Ook zocht de AI actuele brandstofprijzen op.

Mijn beslissing: Ik heb het advies over de sealed class en val overgenomen, omdat dit fouten voorkomt. Ik heb zelf de klassen IceCar, ElectricCar en HydrogenCar geprogrammeerd en de wiskundige TCO-formules opgesteld op basis van de door de AI gevonden prijzen (aannames.md).

2026-10-09 – Database Setup & Repositories

Wat ik vroeg: Hoe ik de overerving van de Car subklassen het beste in de relationele database kon opslaan, en hoe ik complexe joins schrijf in Exposed.

Wat de AI voorstelde: Drie databasestrategieën voor overerving. Voor de database-queries stelde de AI compacte Exposed DSL-syntax voor met ingewikkelde .map {} functies.

Mijn beslissing: Ik koos voor de Single Table Strategy (alles in één tabel) voor de eenvoud. Ik vond de voorgestelde mapping-code van de AI onleesbaar. Ik heb de queries zelf uitgeschreven en expliciete for-loops gebruikt in plaats van .map {}. Om te bewijzen dat ik de query snap, heb ik boven elke Exposed-query in het commentaar de pure SQL-vertaling getypt.

2026-10-12 – Business Logic & Services

Wat ik vroeg: Hulp bij het implementeren van de overlap-controle (zorgen dat een auto niet dubbel geboekt wordt) en het omgaan met datums.

Wat de AI voorstelde: De AI stelde voor om de overlap-check volledig in de database te doen met SQL-tijdfuncties, en wees me op mogelijke problemen met lokale tijdszones.

Mijn beslissing: Ik heb ervoor gekozen de tijd in de hele applicatie in Instant (UTC) vast te leggen. De overlap-logica heb ik niet in de database gestopt, maar in de Reservation domeinklasse, omdat dit een pure bedrijfsregel is.

2026-10-15 – API Endpoints & Ktor Routing

Wat ik vroeg: Hoe ik op een nette manier REST-foutmeldingen (zoals 404 of 400) kan teruggeven in Ktor zonder overal try-catch blokken te schrijven.

Wat de AI voorstelde: Het gebruik van de Ktor StatusPages plugin en het genereren van standaard exception-classes.

Mijn beslissing: Ik heb de setup van StatusPages overgenomen, maar zelf de custom exceptions (NotFoundException, ValidationException) geschreven en gekoppeld aan mijn eigen domein-validaties.

2026-10-19 – Testen & Kwaliteitsborging

Wat ik vroeg: Welke libraries ik het beste kon gebruiken voor unit- en integratietests, en of de AI testdata kon genereren.

Wat de AI voorstelde: Testcontainers en de library Kotest, plus een lijst met mogelijke testgevallen voor de API.

Mijn beslissing: Ik heb Kotest afgewezen en gekozen voor het simpelere kotlin.test in combinatie met een in-memory H2 test-database, omdat dit beter aansluit bij de lesstof. Ik heb de AI wel opdracht gegeven om de boilerplate dummy-data (zoals test-gebruikers en fake adressen) voor me te genereren, waarna ik zelf de Assert-logica heb geprogrammeerd.

2026-10-22 – Beveiliging en Authenticatie

Wat ik vroeg: Best practices voor inloggen en JWT in Ktor.

Wat de AI voorstelde: De AI genereerde een voorbeeld met BCrypt-hashing, JWT-tokens die 24 uur geldig bleven, en geen limieten op inlogpogingen.

Mijn beslissing: Ik heb dit voorstel verworpen op basis van informatie van de gastles. Ik heb de AI specifiek gevraagd hoe ik Argon2id moest implementeren. Daarna heb ik zelf de tokens strak beperkt tot 15 minuten en een Rate Limiter-plugin toegevoegd.

2026-10-24 – Kwaliteitsslag & Refactoring

Wat ik vroeg: Ik vroeg aan de AI wat hij van mijn codebase vond en of hij nog feedback had op de structuur.

Wat de AI voorstelde: De AI gaf als feedback dat de Kotlin-code veel korter en compacter opgeschreven kon worden. Hij stelde voor om functies in te korten met 'expression bodies' (=), impliciete variabelen (it in lambda's) en korte null-checks (!! of ?.let).

Mijn beslissing: Ik heb dit voorstel nadrukkelijk afgewezen. Ik vond deze compacte schrijfwijze totaal niet leesbaar. Ik heb er bewust voor gekozen om de code expliciet en overzichtelijk te houden. Ik heb de voorgestelde one-liners genegeerd en expres overal de functies uitgeschreven met gewone 'block bodies' ({ ... }), waarbij ik de return netjes onderaan heb gezet om de leesbaarheid te vergroten. Ook heb ik snelle null-checks uitgeschreven in duidelijke if-statements en lambda-variabelen een expliciete naam gegeven. Ik heb deze refactoreerslag zelf doorgevoerd en daarna geverifieerd via mijn tests dat de logica 100% intact was.

2026-10-25 – Code Integratie, Documentatie & Oplevering

Wat ik vroeg: Hulp bij een vastgelopen Git-situatie, het updaten van dependencies, en het genereren van een studiegids.

Wat de AI voorstelde/deed: De AI hielp me veilig mijn branches te mergen naar develop na een conflict. Toen ik mijn eigen code uit Periode 1 (DTO's, application.conf) toevoegde, hielp de AI me met het gelijktrekken van de package-namen en wees me op missende dependencies in libs.versions.toml.

Mijn beslissing: Na de succesvolle integratie liepen alle tests door. Als bewijs dat ik de architectuur volledig beheers, heb ik de AI gevraagd een studiegids-PDF te genereren waarin alle in de code gebruikte Kotlin-concepten worden toegelicht. Ik heb deze gids gecontroleerd en geverifieerd dat ik elk beschreven patroon zelf kan uitleggen.